

明日から使える!海外コンペで評価された アクセシビリティ実装ガイド



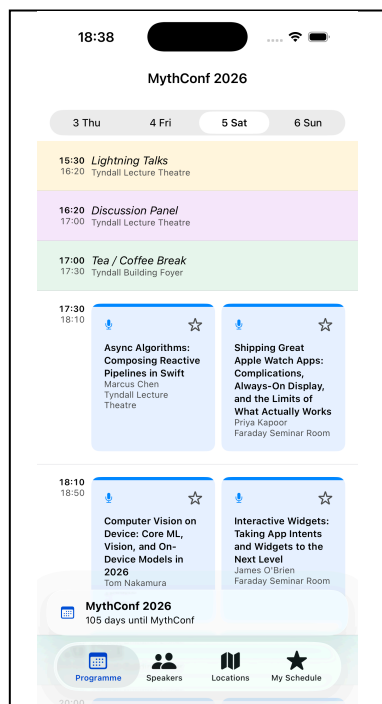
Sansan株式会社
#akidon0000 (あきどん)

誰もが・どんな状況でも、必要とする情報に辿り着けること——それがアクセシビリティ対応の本質だと私は考えています。視覚や聴覚といった身体特性はもちろん、画面の向きやデバイスの違いまで、利用者の置かれた状況はさまざまです。いかなる状況であっても情報に辿り着けるようにする。アクセシビリティ対応は一部のユーザーだけのための特別な対応ではなく、すべての人の使いやすさに直結する「アプリの実装品質」と捉えると良いかもしれません。

iOSには、VoiceOverやDynamic Type、Voice Controlといった強力な支援技術が標準で備わっています。これらは、開発側が意識して対応してこそ、その力を最大限に発揮します。さらに、タップ領域の広さやコントラストのように、標準の支援技術だけでは満たせず、実装側でしか担保できない領域もあります。

本記事では、著者が iOSDevUK カンファレンスの Accessibility Challenge コンペ で優勝した実装を元に、AppleのHuman Interface Guidelinesに沿ってアクセシビリティの解説をします。

題材資料:iOSDevUK Accessibility Challenge



iOSDevUK Accessibility Challengeは、MythConf という架空カンファレンスアプリを題材にアクセシビリティを向上を競うコンペティションです。このアプリはMProgramme、Speakers、Locations、My Scheduleの4タブを持ち、アクセシビリティの観点で改善できる箇所が多数存在していました。

最初にVoiceOverをオンにして題材アプリを触ると、セッションカードは断片的な単語を何度も読み上げ、地図へ入ると操作に迷い、最大文字サイズでは会場名が画面からこぼれました。見た目には完成しているアプリが、使い方を変えた瞬間に別物になったのです。

誰もが・どんな状況でも、必要とする情報に辿り着けること。このアプリを、画面を見なくても、細かなタップが難しくても、必要な情報へ辿り着ける状態に直す——それが今回挑戦したアクセシビリティコンペでした。

優勝した際は、主催のRobin 氏から次の評価をいただきました。

We had some great pull requests, but his was our favorite.

図 1: MythConf の Programme 画面

Appleはアクセシビリティ対応のガイドラインを、利用者の特性に応じて大きく5つの領域に分けて定義しており、これは今回の題材としたコンペの評価軸でもありました。本記事も主にこの5つを地図として進めますが、これがすべてではなく、ここに収まりきれない観点も存在します。

- **Vision（視覚）**：見えづらくても情報が伝わるように。
- **Mobility（身体機能）**：細かな操作が難しくても扱えるように。
- **Cognitive（認知）**：迷わず理解できるように。
- **Hearing（聴覚）**：聞こえなくても気づけるように。
- **Speech（発話）**：声を使わなくても操作できるように。

Vision | 文字サイズに追従する:Dynamic Type

文字サイズの追従は、アクセシビリティ対応の基本といえるでしょう。ユーザーが設定アプリで文字サイズを上げると、`.body`などのテキストスタイルを使っている箇所は自動で拡大されます。このとき考慮すべきなのが、**レイアウトの変化**と**情報量の変化**の2つです。

レイアウトの変化に対応する

文字サイズが大きくなった際の簡単な対策は「横に収まらなければ縦に積む」ことです。ここで使うのが `ViewThatFits` です。 `ViewThatFits` は与えた候補を上から試し、収まる最初のものを採用します。文字サイズの境界（AX1）を待たず、実際にはみ出した瞬間に縦積みへ切り替わるのがポイントです。

`ViewThatFits`は、`HStack`や`VStack`などビューが変化したとしてもビューが作り直されることはなく、パフォーマンスの低下を防ぐことができます。仮に `if / else` の各分岐で実装した場合、サイズが境界を跨ぐたびにビューが丸ごと作り直され、パフォーマンスも低下していただしょう。

```
// 時刻と会場：横に入らなければ自動で縦積みに切り替わる
ViewThatFits(in: .horizontal) {
    HStack {
        Label(session.timeRange, systemImage: "clock")
        Spacer()
        NavigationLink(value: locationID) { locationLinkLabel }
    }
    VStack(alignment: .leading) {
        Label(session.timeRange, systemImage: "clock")
        NavigationLink(value: locationID) { locationLinkLabel }
    }
}
```

Pre-Conference Workshop: SwiftUI Architecture in Practice

🕒 22:00 – 00:00 📍 Tyndall Lecture Theatre >



Sarah Thornton

Sarah is a senior iOS engineer at a >

Pre-Conference Workshop: SwiftUI Architecture in Practice

🕒 22:00 – 00:00

📍 Tyndall Lecture Theatre >

図2: 横に収まる場合: `HStack`

図3: 収まらない場合: `VStack`へ

文字以外もサイズ変化に追従する @ScaledMetric

文字サイズに追従するのは「文字」だけではありません。アイコンのサイズ、余白、そして後述するタップ領域も変化させる必要があります。その際には、@ScaledMetric が活用できます。

```
@ScaledMetric private var starIconSize: CGFloat = 18
@ScaledMetric private var tapButtonSize: CGFloat = 44
```

情報量の変化に対応する

lineLimit を使って行数を固定にしている場合、文字サイズによって文字の表示量が変わりユーザーがその画面で得られる情報量に変化が発生する懸念があります。その際には文字サイズに応じて行数を変化させることも検討してみると良いかもしれません。@Environment(\.dynamicContentSize) を見て、見切れを減らしつつ通常時のレイアウトも保つことが可能です。

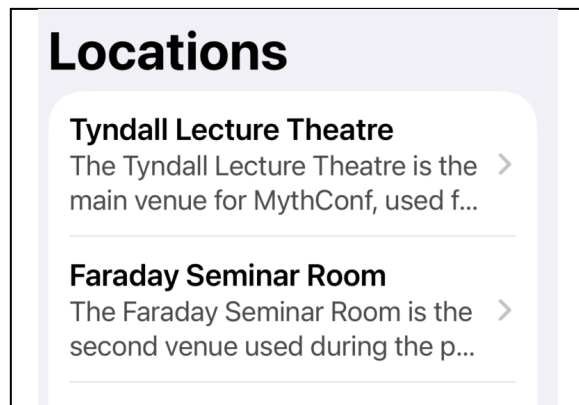


図4: 文字サイズ最小 (xSmall) での表示

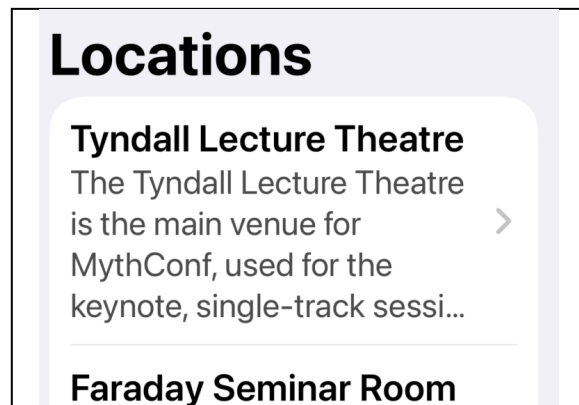


図5: 最大サイズ (AX5) では行数を増やして見切れを防ぐ

```
extension View {
    /// Accessibility サイズ (AX1+) では行数を増やして見切れを防ぐ
    func allyLineLimit(_ standard: Int, extra: Int = 2) -> some View {
        modifier(A11yLineLimitModifier(standard: standard, extra: extra))
    }
}

private struct A11yLineLimitModifier: ViewModifier {
    @Environment(\.dynamicContentSize) private var dynamicContentSize
    let standard: Int
    let extra: Int

    func body(content: Content) -> some View {
        content.lineLimit(dynamicContentSize.isAccessibilitySize ? standard +
        extra : standard)
    }
}
```

文字サイズの変化を許容できない際のベストプラクティス

タブバーやツールバーのように、デザイン上どうしても文字・アイコンを拡大できない要素もあります。とはいえ「小さいまま放置」では、拡大表示を必要とするユーザーがその要素を読めません。そこで活用できるのが **Large Content Viewer** です。

Large Content Viewerは、アクセシビリティサイズ設定時に対象要素を長押しすると、画面中央に拡大したアイコンとラベルをHUDとしてオーバーレイ表示する仕組みです。自前で作ったカスタムコントロールは自動では対応しないため `.accessibilityShowsLargeContentViewer` で拡大時に見せる内容を明示的に渡す必要があります。

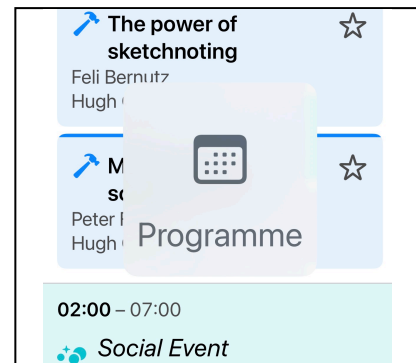


図6: 長押しして拡大ラベルをHUD表示（レイアウトは維持）

Vision | 画面を見ずに音で操作する:VoiceOver

VoiceOverは、画面を見ずにアプリを操作するためのスクリーンリーダーです。ただし「よしなに読み上げてくれる便利機能」ではなく、何も設計しなければボタンはただ「ボタン」としか読まれません。音は目に比べて情報密度が圧倒的に低いため、何を補い何を読み上げないかを取捨選択し、限られた音の帯域を最大限に活かすことが対応の肝になります。

- **URLは「種類」を先に伝える**：スピーカー詳細にあるGitHubやXなどのプロフィールリンクは、URLをそのまま読ませると「https://github.com/akidon0000」と聞き取りづらくなります。ホストとパスを解析し、`accessibilityLabel`を「GitHub アカウント、akidon0000」と組み立てれば、情報量を減らし伝えることが可能です。
- **バラバラの要素は1つにまとめる**：1件のセッションを表すカード（タイトル・スピーカー・会場などが集まったもの）は、要素ごとにバラバラ読み上げるより、`.accessibilityElement(children: .combine)` で1つにまとめた方が圧倒的に速く理解できます。
- **読み上げ順は「結論から」**：`.accessibilitySortPriority` で順序を制御します。MythConfのカードでは「種別→タイトル→スピーカー→会場→お気に入り」の順にし、長くなりがちタイトルより先に「これはWorkshopだ／Talkだ」と分かるようにしました。聞き手の負担が小さくなります。

```
VStack(spacing: 0) { // TODO 上記のやつとコードの内容が乖離
    Image(systemName: iconName)
        .accessibilityHidden(true) // 装飾は読み上げから除外

    FavouriteButtonView(talk: talk)
        .accessibilitySortPriority(0) // 最後に読む

    NavigationLink(value: reference) {
        /** HogeHoge */
    }
    // 「種別, タイトル, by スピーカー, at 会場」を1フレーズに
    .accessibilityLabel("\(kind), \(title), by \speakers), at \(location)")
    .accessibilitySortPriority(1) // 先に読む
}
.accessibilityElement(children: .contain) // コンテンツをそれぞれ独立した操作に
```

状態が変化した時に、変化後の状態を知らせる

お気に入りのように状態がトグルする要素は、`accessibilityLabel` を現在の状態に応じて出し分けます。こうすると、トグル後にVoiceOverが読み直され、最新の状態を耳で確認することができます。

文言そのものにもこだわしましょう。状態（オン／オフ）ではなく「押したら何が起きるか」を伝えるべきです。VoiceOverはボタンに自動で「ボタン」と読み添えるため、未追加は「Favourite, ボタン」と読まれ、**押せば Favourite できると直感的に伝わります**。追加済みの場合は `"Remove session from favourites"` とすることで、短くても意味が通るようになります。

加えて `.sensoryFeedback(.success, trigger: isFavourite)` で、音が聞こえない環境でも触覚で切り替わりが伝わります。

```
Button {
    isFavourite ? viewModel.removeFavourite(talk: talk)
                : viewModel.addFavourite(talk: talk)
} label: {
    Image(systemName: isFavourite ? "star.fill" : "star")
        .foregroundColor(isFavourite ? .yellow : .textSecondary)
}
.accessibilityLabel(isFavourite ? Text("Remove session from favourites")
                    : Text("Favourite"))
.sensoryFeedback(.success, trigger: isFavourite)
```

地図はApple Mapsに丸投げする

SwiftUIの `Map` は、VoiceOverではピンやズームの読み上げが事実上操作不能で、手が止まってしまいます。そこで **VoiceOver上では地図を「Apple Mapsを開く1つのボタン」に置き換え**ました。`accessibilityRepresentation` を使えば、見た目はインタラクティブな地図のまま、支援技術にだけ別の表現を渡せます。晴眼ユーザーには通常の地図、VoiceOver／Switch Controlユーザーには使い慣れたApple Mapsへ。どちらの体験も犠牲にせず、地図は純正に任せるのが最善だと考えています。

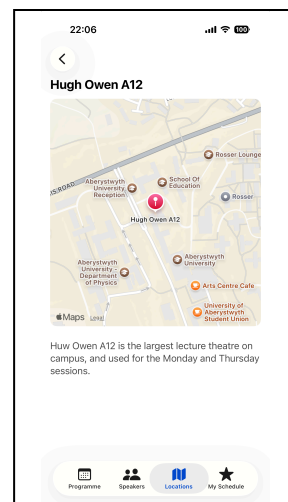


図7: 会場詳細の地図（タップでApple Mapsへ）

```
LocationMapView(location: location, coordinate: coordinate)
.accessibilityRepresentation {
    Button("") { openInMaps() }
        .accessibilityLabel(Text("Open in Maps: \(location.name)"))
        .accessibilityInputLabels(["Map", "Directions", location.name])
}
```

Vision | 色による視認性

コントラスト比はコンパイルエラーも出ず気づきにくいいため、Xcode付属の **Accessibility Inspector** (Xcode → Open Developer Tool) で実測しましょう。指標には **Web Content Accessibility Guidelines (WCAG)** が広く使われており、本文テキストは **4.5:1**、UI部品や大きな文字は **3:1** 以上が推奨されています。

システムカラーだからOKではない

`.foregroundColor(.secondary)`、便利ですよー——実測するまでは。システムの `.secondary` は白背景で約 **3.44:1** と、本文テキストのAA (4.5:1) に届きません。`.yellow`・`.mint` も白背景では約 **1.5:1**・**約2.4:1** と、UI部品の下限3:1の半分しかありません。「Appleの色だから大丈夫」は必ずしも成り立たないのです。今回はコンテストということもあり色味の印象を保ったままAA以上を満たす独自トークンへ置き換えましたが、実際にどこまで対応すべきかはチームで共通認識をそろえておくのが良さそうです。

状態は色だけでなく「形」でも伝える

状態を色だけで示すのは避けたいデザインです。下の星は、黄色を判別しづらいユーザーには状態が伝わりづらい可能性があります。そこでオン時は塗りつぶしに、`sparkles` を重ねました。アニメーションやハプティクスで補うのも手です。



図8: カラー・オフ



図9: カラー・オン



図10: 白黒・オン

Vision & Cognitive | 「押せる」と気づかせる

「ここはタップできる」という手がかりを、形や色で明確に示すことも非常に重要です。

- **リストのセル**: 行末に `chevron.right` を置き「タップで遷移する」と示す。
- **本文中のリンク**: 下線や色で本文と差別化する。
- **ボタ的な要素**: 下線が使いにくい箇所は円形背景付きの矢印 (`arrow.up.right.circle.fill` など) で「押せる」と示す。
- **外部アプリ・サイトへ飛ぶ場合**: 行末に `arrow.up.right.square` を添え「別アプリ／別サイトに飛ぶ」と明示する。
- **URL リンク**: 下線での差別化が難しければ青系の色で示すのも手。ユーザーに新たな学習を強いず、青を見分けづらい色覚特性の割合も少ないため。

URLやボタンの色が青である背景は、[なぜデフォルトが青色！？ Tint Colorの理由に迫る by akidon0000 (iOSDC Japan 2024 ルーキーズLT)] で詳しく話しています。

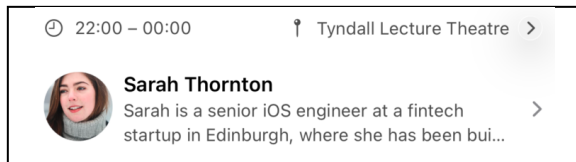


図11: chevronで「タップで遷移」を予告

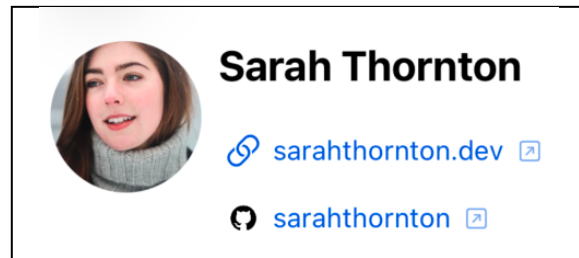


図12: 外部リンクは矢印アイコンで予告

その他の領域でやったこと (Mobility / Cognitive / Hearing / Speech)

残りの領域も、これまでと同じ「OSに正しい情報を渡し、実装でしか担保できない部分を補う」発想で一通り対応しました。要点だけまとめます。

- **Mobility** | **タップ領域は44×44pt以上**：アイコンが小さくても当たり判定は44pt確保。
`.frame(minWidth: 44, minHeight: 44)` に `.contentShape(.rect)` を添えて透明部分まで反応させ、`@ScaledMetric` で拡大時に一緒に広げる。
- **Mobility** | **ジェスチャには必ず代替を**：日付切り替えはピッカー+左右スワイプ、`NavigationStack` の画面端スワイプ戻るも有効化。「ジェスチャでしかできない操作」を作らない。
- **Cognitive** | **語彙と表示の一貫性**：同じ操作は同じ言葉（お気に入り=Favourite / Save、地図=Open in Maps）。タブは「3 Thu」表記、検索ゼロ件は `ContentUnavailableView` で明示。
- **Cognitive** | **Reduce Motion**：`@Environment(\.accessibilityReduceMotion)` がオンなら自動アニメーション（並行セッションの自動切り替えやMarqueeスクロール）を止めて静的表示に。省略時もVoiceOverには全文を渡す。
- **Hearing** | **音に触覚を添える**：`.sensoryFeedback(.success, trigger:)` で、音が聞こえなくても操作の成否が伝わる。
- **Speech / Switch Control** | **声もタッチも使わず操作できる**：⌘1〜4・⌘F・⌘Dなどのショートカットを、画面に出ない隠しボタン（`.background + .hidden()`）で実装。地図を単一ボタンに畳む・要素をまとめる工夫も、スイッチ操作の手数削減にそのまま効く。

枠を超えたUX向上としても、Wi-Fiが不安定な会場を想定した**オフライン地図**（`MKMapSnapshotter` でライト/ダーク両方をキャッシュし、`NetworkMonitor` がオフラインを検知したらキャッシュ画像へ差し替え）や、`.ultraThinMaterial`・実行時の多言語切り替えなどを、同じ「全員の底上げ」の発想で実装しました。

Voice Controlは「言いそうな言葉」を全部登録する

その中でも少し掘り下げたいのがVoice Control対応です。Voice Controlは画面の表示文字で要素を呼び出しますが、ユーザーが必ずしも表示どおりに言うとは限りません。そこで `.accessibilityInputLabels` に **言い換えの候補** をまとめて登録しておきます。

```
// お気に入りの星：英米スペル違いや同義語をまとめて登録
.accessibilityInputLabels([
    "Favourite", "Favorite", "Star", "Bookmark",
    "Save", "Add to schedule", "Remove from schedule"
])
```

英米のスペル違い（Favourite / Favorite）から「Star」「Bookmark」「Save」といった同義語までカバーすると、ユーザーは自分の自然な言葉でボタンを押せます。

同じ発想は日付ピッカーにも効きます。画面には「3 Thu」とだけ出ていても、ユーザーは「3 Thursday」「Thursday」「Day 1」など様々に呼びます。そこで表示どおりの呼び方に加え、フル曜日や「Day N」も候補として登録しておきます。

```
// 日付タブ：見たまま + 省略形 + 「Day N」を全部受け付ける
.accessibilityInputLabels([
    "3 Thursday", "3 Thu", "Thursday", "Thu", "Day 1"
])
```

SNSリンクには「Tweet」やBlueskyの俗称「Skeet」まで登録しました。言いそうな言葉を先回りして拾うほど、音声操作は滑らかになります。

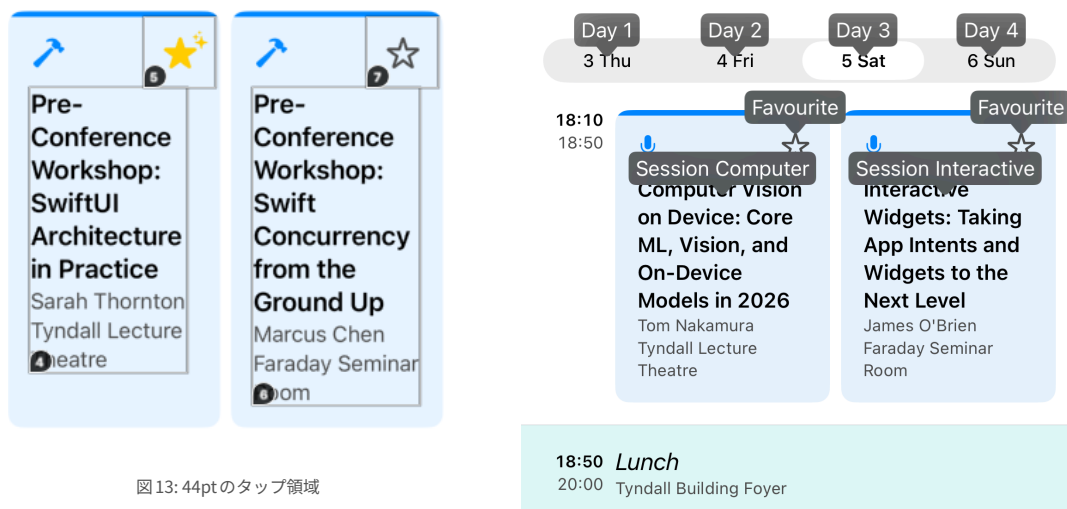


図 13: 44pt のタップ領域

図 14: Voice Control の呼び名表示

最後に

会社はマーケットを見て動いています。もちろん特定のユーザーのために数十万コストをかけて対応するのはコスパに見合っていないかもしれませんが、しかし、アクセシビリティ対応の術を知っていると知らないでは大きく違うと考えています。

アクセシビリティ対応は、特別な誰かのための機能追加ではなく、**実装品質そのもの**です。OSが用意した支援技術に「正しい情報を渡す」こと、そしてコントラストやタップ領域のように **実装でしか担保できない領域**を意識すること。本記事で挙げた施策は、どれも明日からあなたのアプリに1つずつ取り入れられるものばかりです。

そして極論を言えば、**画面回転への対応や、iPhone・iPad・Vision Proといった各デバイス・各サイズへの対応もアクセシビリティです**。横向きで固定されたり、特定のデバイスでレイアウトが破綻したりすれば、その向き・その端末を使う人は情報にたどり着けません。冒頭で述べた「誰もが・どんな状況でも、必要な情報に辿り着けること」へ、Dynamic TypeやVoiceOverもすべて地続きでつながっています。

まずは自分のアプリでVoiceOverをオンにし、Dynamic Typeを最大まで上げて、端から端まで触ってみてください。きっと、直したくなる場所が見つかります。



本記事で解説した実装PR



iOSDevUK26 アプリ (App Store)



#akidon0000 (あきどん)

アドバイザー：@mtj_j